Módulo profesional: Desarrollo Web en Entorno Servidor (DWES)

Capítulos: 2 bis y 3
Duración estimada: 12–14 horas (4–5 sesiones)
Resultados de aprendizaje relacionados:
- **RA1:** Selecciona arquitecturas y tecnologías de programación web en entorno servidor.
- **RA5:** Desarrolla aplicaciones web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.

Capítulos 2 bis y 3 – Spring Boot y Controladores REST

La navegación lateral de la página refleja automáticamente las secciones siguientes y evita mantener a mano una segunda tabla de contenidos.

Capítulo 2 bis – Introducción a Spring Boot

¿Qué es Spring Boot?

Spring Boot es una extensión del *Spring Framework* que facilita la creación de aplicaciones web modernas y APIs REST con: - **Autoconfiguración**: detecta dependencias y configura el entorno automáticamente. - **Starters**: conjuntos de dependencias preconfigurados (web, data, validation...). - **Servidor embebido**: incluye Tomcat/Jetty para ejecutar directamente. - **Convención sobre configuración**: usa valores por defecto sensatos. - **Ejecutables autónomos**: se empaquetan en un único `.jar`.

Estructura de un proyecto típico

```
src/
└─ main/java/com/ejemplo/app/
├─ Application.java
├─ web/           ← Controladores REST
├─ domain/        ← Clases de dominio
├─ service/       ← (se verá más adelante)
└─ repo/          ← (más adelante con JPA)
resources/
└─ application.yml
```

Autoconfiguración y starters

- `spring-boot-starter-web`: Controladores REST, JSON, servidor embebido.
- `spring-boot-starter-validation`: Validación de datos con `@Valid`.
- `spring-boot-starter-test`: Entorno de pruebas integrado.
- `spring-boot-starter-data-jpa`: (se verá más adelante) Persistencia con JPA/Hibernate.

Ejemplo paso a paso – Mini API “Tasks”

> Objetivo: crear una pequeña API REST funcional en memoria, con operaciones básicas GET y P0ST, para entender cómo responde Spring Boot.

1. Clase principal

```java
package com.example.tasks;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

2. Objeto de dominio

```java
package com.example.tasks.web;

public class Task {
    private Long id;
    private String title;
    private boolean done;

    public Task(Long id, String title, boolean done) {
        this.id = id;
        this.title = title;
        this.done = done;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public boolean isDone() { return done; }
}
```

3. Controlador básico

```java
package com.example.tasks.web;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.ArrayList;
import java.util.List;

@RestController
@RequestMapping("/api/tasks")
public class TaskController {

    private final List<Task> tasks = new ArrayList<>();
    private long nextId = 1;

    @GetMapping
    public List<Task> list() {
        return tasks;
    }
```

```java
    @PostMapping
    public ResponseEntity<Task> create(@RequestBody Task newTask) {
        Task saved = new Task(nextId++, newTask.getTitle(), false);
        tasks.add(saved);
        return ResponseEntity
                .created(URI.create("/api/tasks/" + saved.getId()))
                .body(saved);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Task> get(@PathVariable Long id) {
        return tasks.stream()
                .filter(t -> t.getId().equals(id))
                .findFirst()
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

Conceptos introducidos:

- `@RestController`: devuelve datos JSON, no vistas.
- `@RequestMapping("/api/tasks")`: define el punto de entrada de la API.
- `@GetMapping`, `@PostMapping`: métodos HTTP GET/POST.
- `@RequestBody`: convierte el JSON de entrada en objeto Java.
- `@PathVariable`: obtiene parámetros desde la URL.
- `ResponseEntity`: devuelve respuesta completa (estado, cabeceras, cuerpo).

4. Probar la API

```
./mvnw spring-boot:run
```

Endpoints:

- `GET /api/tasks` → lista tareas (vacía al inicio)
- `POST /api/tasks` → crea una nueva tarea
- `GET /api/tasks/{id}` → obtiene una tarea concreta

5. Qué aprendemos

1. Cómo arranca una aplicación Spring Boot.
2. Cómo crear un controlador REST.
3. Cómo devolver datos en JSON.
4. Qué hace `ResponseEntity`.
5. Cómo manejar rutas y parámetros.

Clases vs Records: entidades y DTOs

Uso	Tipo	Mutabilidad	Constructor vacío	Recomendado para
Entidades	`class`	Mutable	Sí	Persistencia con JPA
DTOs	`record`	Inmutable	No necesario	Transporte de datos, respuestas REST

- **Entidades:** deben ser mutables (Hibernate necesita modificar y construir instancias).
- **DTOs:** ideales como `record`, son compactos e inmutables.

```java
public record TaskDTO(Long id, String title, boolean done) {}
```

Conclusión:

- Usa `class` para entidades JPA.
- Usa `record` para DTOs o respuestas REST.

Capítulo 3 – Controladores y vistas (adaptado a REST)

3.1 Características de los controladores

Un **controlador** recibe peticiones HTTP, ejecuta la lógica correspondiente y devuelve una respuesta. En APIs REST, suele devolver **datos JSON**, no páginas HTML.

- Se anota con `@RestController`.
- Se organiza por rutas con `@RequestMapping`.
- Cada método responde a un verbo HTTP.

3.2 Tipos de controladores

- **`@RestController`** → devuelve JSON (APIs REST).
- **`@Controller`** → devuelve vistas (Thymeleaf u otros motores).

3.3 El paso de datos a la vista

En REST, la "vista" es el **cuerpo JSON** de la respuesta. Spring convierte automáticamente objetos Java en JSON con **Jackson**.

3.4 Parámetros de URL

- `@PathVariable` → parte dinámica de la ruta (`/users/{id}`)
- `@RequestParam` → parámetros en query (`?page=1`)
- `@RequestHeader` → cabeceras HTTP
- `@RequestBody` → cuerpo JSON de la petición

3.5 Retorno de métodos

- Devolver directamente un objeto → `200 OK`.
- Usar `ResponseEntity` → controlar código, cabeceras y cuerpo.

```
@GetMapping("/{id}")
public ResponseEntity<Task> get(@PathVariable Long id) {
    return tasks.stream()
            .filter(t -> t.getId().equals(id))
            .findFirst()
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
}
```

3.6 Gestión de errores

Se usa `@ControllerAdvice` + `@ExceptionHandler` para capturar excepciones globalmente y devolver un JSON uniforme de error.

3.7 ResponseEntity y códigos de estado HTTP

ResponseEntity<T> representa toda la respuesta HTTP:

- **status** (200, 201, 404…)
- **headers** (cabeceras)
- **body** (cuerpo JSON)

Código	Significado	Uso habitual
200	OK	Petición correcta
201	Created	Recurso creado (POST)
204	No Content	Eliminación o actualización sin cuerpo
400	Bad Request	Error de validación o formato
404	Not Found	Recurso inexistente
409	Conflict	Conflicto de estado o duplicado
422	Unprocessable Entity	Error semántico

Ejemplo:

```
@PostMapping
public ResponseEntity<TaskDTO> create(@RequestBody @Valid CreateTaskDTO dto) {
    Task saved = service.create(dto);
    URI loc = URI.create("/api/tasks/" + saved.getId());
    return ResponseEntity.created(loc).body(new TaskDTO(saved.getId(), saved.getTitle(), save
}
```

Otras formas:

```
ResponseEntity.ok(body);                      // 200 OK
ResponseEntity.status(HttpStatus.CREATED);    // 201 Created
ResponseEntity.noContent().build();           // 204 No Content
ResponseEntity.badRequest().body(errorMsg);   // 400 Bad Request
```

Guion práctico y entregable

1. Crear `HelloController` con `GET /api/hello` → `{ "message": "Hola DWES" }`.
2. Añadir `GET /api/tasks`, `GET /api/tasks/{id}`, `POST /api/tasks`.
3. Implementar `ResponseEntity` correctamente (201 en creación, 404 en no encontrado).
4. Añadir `@ControllerAdvice` con manejo básico de errores.
5. Importar colección en Insomnia o Postman para probar.

Entregable mini:

- Rama o commit `ud2-cap3`.
- 3 endpoints funcionales.
- Colección de pruebas en Insomnia/Postman.
- Respuestas con códigos adecuados.

Próximos pasos: proyecto completo Battleship API

En las próximas unidades desarrollaremos un **proyecto completo (Battleship API)** aplicando todos los conceptos:

- Controladores REST organizados.
- Capa de servicios y repositorios.

- Entidades JPA, relaciones y validación.
- Seguridad, JWT y roles.
- Documentación con Swagger.

Este proyecto servirá como eje central del módulo, integrando todo lo aprendido en Spring Boot hasta la persistencia y seguridad.
